1-2 3.1 Constraint Satisfaction Problems (CSPs) (mit diskreten Wertebereichen) (und vorwiegend binären Constraints)

CSPs sind eine besondere Art von Suchproblemen:

- Formalere Repräsentation
- Backtrackingsuche über das Belegen von Variablen
- Lokale Propagierung (Forward Checking) hilft, den zu expandierenden Suchbaum zu verkleinern. First-Fail-Heuristiken beschleunigen die Suche in den meisten Fällen.
- 2-Konsistenz beschneidet den Suchraum a priori.
- Der Ausblick beinhaltet den Einstieg in das Thema „lokale Suche“

3 Ein Constraint Satisfaction Problem (CSP) besteht aus Domainvariablen und Constraints

Variablen in CSPs werden auch *Constraintvariablen* oder *Domainvariablen* genannt.

D steht für Domain. In diesem Kurs betrachten wir ausschließlich endlich aufzählbare Wertebereiche (so genannte *Finite Domains*), z.B. die Farben {rot, grün, blau} oder einen Zahlenbereich {1,2,3,4}.

Ein Constraint drückt eine Beziehung zwischen 1, 2 oder mehr Variablen aus, indem es die möglichen (konsistenten) Wertetupel benennt. Dies kann z.B. in einer Tabelle geschehen (unten auf der Folie), die für jede Variable eine Spalte hat und in der sämtliche konsistente Wertetupel enthalten sind (ähnlich einer Datenbankrelation). Üblicherweise haben wir ein Geflecht von Beziehungen zwischen Variablen. Dies führt dann zu einem ConstraintNETZ.

Die Namen der Variablen sind beliebig. Wir notieren Variablen über die Domain mittels $D_x = \{ 1, 2, 3 \}$ („Domain von x sind die Wert 1, 2, 3“). Möglich ist auch die direkte Variablendefinition mittels $x \in \{ 1, 2, 3 \}$.

Im Folgenden betrachten wir vorzugsweise 2-stellige Constraints (binäre Constraints) (der Art „a=b“ oder „a≠b“ oder „a>b+2“). Ein Beispiel für ein einstelliges Constraint wäre „a>0“.

4 Sudoku als Constraint Satisfaction Problem

81 „Felder“-Variablen, jeweils mit Wertebereich {1,2,3,4,5,6,7,8,9}. Die Variablen werden wie im Schach mit Buchstaben für die Spalten und Zahlen für die Zeilen notiert. Unten links liegt die Variable v(a,1), oben rechts v(i,9).

Paarweise Ungleichheitsconstraints zwischen allen Variablen, die im selben Quadranten, in derselben Spalte oder in derselben Zeile liegen: v(a,1)≠v(a,2), v(a,1)≠v(a,3), ..., v(a,1)≠v(c,3), ..., v(a,1)≠v(i,1).

Statt die Constraints alle einzeln aufzulisten, verwendet man üblicherweise ein „Makro“ namens „all-different“. Alle Elemente eines all-different-Constraints sind Constraintvariablen, die paarweise ungleich sind: all-different({v(a,1), v(a,2), ..., v(a,9)}), all-different({v(a,1), v(b,1), ..., v(i,1)}), all-different({v(a,1),v(a,2),v(a,3),v(b,1),v(b,2),v(b,3),v(c,1),v(c,2),v(c,3)}), ... (Doppeltnennungen vergrößern die Problemkomplexität nicht).

Bezug zu Suchproblemen:

- Startzustand = Anfangskonfiguration; Aktion = trage Zahl in freies Feld ein; Zieltest = kein freies Feld und alle Constraints erfüllt.
- CSPs sind stärker formalisiert als allgemeine Suchprobleme: Der Zustand wird durch Domainvariablen beschrieben. Aktionen benötigen wir nicht, die sind implizit vorhanden, indem wir eine Variable mit einem Wert ihrer Domain belegen. Auch der Zieltest entfällt, er ist implizit gegeben durch: Alle Variablen belegt, kein Constraint verletzt.

Komplexitätsbetrachtungen für das Sudoku:

- 81 Felder, davon 28 belegt. Das ergibt eine Problemraumgröße von 9^{53} Zuständen.

- Der Zieltest ist extrem aufwändig: Jede Variable tritt in Insgesamt 20 Ungleichheitsconstraints auf: mit jeder Variable in derselben Zeile, Spalte und Quadranten. (8 Constraints für die Zeile, 8 Constraints für die Spalte und 4 weitere Constraints für die Variablen im Quadrat (die bereits auf der Spalte oder Zeile liegenden nicht mitgerechnet). Bei 81 Variablen macht dies 20x81/2 = 810 Constraints. (Geteilt durch 2 wegen der Symmetrie: v(a,1)≠v(a,2) ist dasselbe Constraint wie v(a,2)≠v(a,1)).

Bemerkungen:

- Es wäre auch möglich, die bereits ausgefüllten Felder als Variablen mit 1-stelligen Wertebereichen zu definieren und dafür auf die einstelligen Gleichheitsconstraints zu verzichten: v(a,2):{5}, v(a,5):{9} usw.

5 Graphenfärbung ist ein Standardproblem der KI

2. Beispiel stellt eine Standardanwendung der KI dar: Das Minimum benötigter Farben definiert die chromatische Zahl. Für planare Graphen (z.B. Landkarten) ist die chromatische Zahl 4, wobei in vielen Fällen 3 Farben ausreichen. Der Aufwand zur Bestimmung der chromatischen Zahl liegt in NP. Beispiele für Graphenfärbeprobleme:

- Sudoku ist ein Graphenfärbeproblem mit 81 Knoten und 9 Farben.
- Klausurplanung: Gegeben eine Menge von Modulen mit eingeschriebenen Studierenden. Wie viele Klausurtermine werden benötigt, damit kein Studierender mehrere Klausuren zur selben Zeit schreiben muss? Repräsentation als Graph, in dem jedes Modul durch einen Knoten repräsentiert wird. Eine Kante zwischen 2 Knoten existiert genau dann, wenn mindestens ein Studierender beide Module belegt hat. Die chromatische Zahl des Graphen (minimale Zahl an Farben) entspricht der Zahl der Klausurtermine.
- 2-Färbung = Test ob bipartit.

Besonderheit: Alle Variablen haben denselben Wertebereich und alle Constraints sind Ungleichheitsconstraints.

6 - 9 Job Shop Scheduling als Constraintproblem

Job Shop Scheduling ist Maschinenbelegungsplanung. Jeder Fertigungsauftrag (Job) besteht aus mehreren Bearbeitungsschritten (Tasks), die in bestimmter Reihenfolge auf verschiedenen Maschinen zu erledigen sind. Dabei verbraucht jeder Verarbeitungsschritt Zeit. Die Jobs sind unter Beachtung der Reihenfolge ihrer Bearbeitungsschritte so auf die Maschinen zu verteilen, dass keine Überschneidungen stattfinden. Die Gesamtbearbeitungszeit ist zu minimieren.

Tabelle: 4 Jobs mit 2 bzw. 3 Bearbeitungsschritten unterschiedlicher Dauer (Angabe in Stunden in Klammern) auf 3 Maschinen.

CSP-Definition:

- Variablen: JxMy (x = JobNr, y = MaschinenNr). Insgesamt 10 Domainvariablen.
- Domains: Früheste Startzeitpunkte als Wertebereiche: 0, ... 37 (Summe aller Joblängen, ist sehr konservative Abschätzung).

Zeitliche Abhängigkeiten (vorher-nachher-Beziehungen zwischen den Bearbeitungsschritten eines Jobs werden durch 2-stellige "<"-Constraints ausgedrückt und im Constraintgraph als ungerichtete schwarze Kanten visualisiert:

- J1M1 + 4 ≤ J1M2; J1M2 + 3 ≤ J1M3; usw. (insgesamt 6 derartige Constraints)

Jede Maschine kann nur einen Job (bzw. Task) gleichzeitig ausführen. Zu definieren als **Überlappungsfreiheit** für jedes Paar von Tasks, die auf derselben Maschine stattfinden: Der Startzeitpunkt plus Dauer einer Task x auf Maschine z muss kleiner oder gleich sein dem Startzeitpunkt von Task y auf derselben Maschine (und umgekehrt).

Z.B. das Constraint zwischen J3M2 und J4M2 lautet:

- (J3M2 + 4 ≤ J4M2) v (J3M2 ≥ J4M2 + 6). Insgesamt existieren 12 derartige Constraints, visualisiert im Constraintgraph als rot gestrichelte Linien.

Terminconstraints sind einstellige Constraints wie

- J4M3 +2 ≤ 20 (Job 4 muss spätestens zum Zeitpunkt 20 abgeschlossen sein. Die +2 repräsentiert die Dauer des Jobs 4 auf Maschine 3 entsprechend dem Eintrag in der Tabelle. Im Constraintgraphen als blaue Kanten dargestellt, die einen Knoten mit sich selbst verbinden.

Andere einstellige Constraints könnten den Startzeitpunkt eines Jobs betreffen oder Wartungszeiten der Maschinen definieren.

Bemerkungen:

- Visualisierung als Constraintgraph (bzw. Constraintnetz): Constraintvariablen sind Graphknoten, Constraints sind Kanten. Besonderheit: Einstellige Constraints sind Kanten von einem Knoten in sich selbst.
- Wir erkennen, dass Constraints die Zulässigkeit von Lösungen beschreiben bzw. einschränken. Eine Optimierung (wie die Kostenminimierung bei Suchproblemen) können wir nicht als Constraint formulieren.
- Besonderheit beim Job Shop Scheduling: Zahlreiche arithmetische Constraints.

10 - 11 Quizfrage und Agenda

Überleitung zu den Problemlöseverfahren für CSPs: Welches Suchverfahren ist für CSPs am besten geeignet?

Ansatz zur Beantwortung: CSP als Suchproblem: Wie lauten Startzustand, Aktion und Zieltest? (Am besten klarmachen am Beispiel der Graphenfärbung / Sudoku).

12 Ein CSP wird durch konsistentes Belegen aller Variablen gelöst

Wichtige Begriffe veranschaulicht am Beispiel der Graphenfärbung.

Das Lösen eines CSPs erfolgt durch Zuweisung von Werten an die Variablen. Die korrekte Fachsprache lautet: Wir *belegen* eine Variable mit einem Wert ihrer Domain.

Den Zustand eines CSPs nennt man ebenfalls *Belegung*. Zu Beginn sind (typischerweise) alle Variablen unbelegt (siehe a1) - auf der Folie sind unbelegte Variablen als weiße Knoten dargestellt). Auch die leere Belegung ist eine Belegung! In a2) sind die 3 Variablen a, d und e belegt (mit grün, rot, bzw. grün) und die anderen Variablen noch unbelegt.

Hier wird der Bezug zu Suchproblemen deutlich: Belegung = Zustand. Komplexitätsbetrachtungen sind dann sehr einfach, denn die Zahl der möglichen Belegungen entspricht dem Produkt der um 1 inkrementierten Wertebereichsgrößen (um 1 inkrementiert, weil auch die Nichtzuweisung eines Wertes berücksichtigt werden muss). Im Beispiel auf der Folie (Graphenfärbung mit 3 Farben für 5 Knoten ergibt insgesamt 4^5 = 256 mögliche Belegungen.

Eine *konsistente Belegung* verletzt keine Constraints. Zum Beispiel b), wo alle Constraints zwischen den belegten Variablen a, c und d erfüllt sind. Constraints zwischen unbelegten Variablen spielen hierbei keine Rolle auch nicht Constraints zwischen belegten UND unbelegten Variablen.

Eine *vollständige Belegung* ist ein Lösungskandidat. Hier sind alle Variablen belegt. c) zeigt eine solche Situation, die allerdings nicht konsistent ist, denn das ≠-Constraint zwischen den beiden mit grün belegten, "benachbarten" Variablen a und e ist verletzt. (Und ebenso das Constraint zwischen den blau gefärbten Knoten b und c).

Jede *vollständige und konsistente Belegung* ist eine Lösung für ein CSP. Eine solche Belegung zeigt das Beispiel d).

Im Gegensatz zu Suchproblemen suchen wir keine „optimale“ Lösung. Jede „zulässige“ Lösung ist gleichgut! Und diese Erkenntnis führt auch zur Beantwortung der Quizfrage: Ein geeignetes Verfahren zum Lösen eines CSPs ist die Tiefensuche. Denn wir wollen ja möglichst schnell einen Lösungskandidaten generieren und nicht etwa in die Breite suchen.

Bemerkungen:

- Eine Variable ist entweder unbelegt oder belegt.
- Dx={rot} beschreibt eine Domainvariable mit einstelligem Wertebereich. Diese Variable ist aber noch unbelegt! Auch wenn nur ein einziger Wert zur Wahl steht.

13 CSPs sind Suchprobleme mit uniformen Aktionskosten

Ein CSP ist eine Art "getyptes" Suchproblem:

- Zustand = Belegung (zugewiesene Werte an 0,...,n Variablen). Die unveränderlichen Wertebereiche und Constraints werden üblicherweise außerhalb des Zustands gespeichert. Man kann aber auch alles in den Zustand packen (wie hier dargestellt).
- Startzustand: leere Belegung (auch beim Sudoku: Dort sind die vorgegebenen Werte entweder Variablen mit einstelligen Domains (aber noch unbelegt) oder Variablen mit 9-stelligen Domain plus einem einstelligen Gleichheitsconstraint.
- Aktion = Belege eine Variable mit einem Wert ihrer Domain. Bei 5 Variablen mit jeweils 3-stelligem Wertebereich liegt ein Verzwei-gungsgrad von 15 vor. Man kann die Variablen aber auch in einer fixen Reihenfolge belegen (dann bekommen wir bei der Expandierung immer 3 Nachfolgezustände, also b=3).
- Zieltest: erfüllt, wenn alle Variablen belegt und kein Constraint verletzt. Eine vollständige Belegung ist nur in Blattknoten vorhanden (Baumtiefe = Anzahl der Constraintvariablen).

Weil ein CSP ein Suchproblem ist, kann es mit den bisher bekannten uninformierten Suchverfahren gelöst werden. D.h., jeder Algorithmus, der ein Suchproblem löst, löst auch ein CSP.

14 Besonderheiten beim Lösen von CSPs

Ziel: Bei CSPs sucht man irgendeine Lösung. Häufig gibt es mehrere Lösungen (siehe Graphenfärbung), es gibt aber keine „beste“ Lösung, denn CSPs sind keine Optimierungsprobleme. Wir können nur zulässige Lösungen spezifizieren, beispielsweise beim Job Shop Scheduling späteste Endtermine.

Stundenplanerstellung oder auch Einsatzplanung oder Schichtplanung oder das Erstellen von Fahrplänen bei der Bundesbahn sind CSPs mit so genannten Softconstraints. Normale "harte" Constraints (z.B. keine 2 Veranstaltungen zur selben Zeit im selben Raum) müssen erfüllt werden. Aber Softconstraints (z.B. Sprachkurse möglichst im Sprachlabor; Lehrer Lampe möchte nur früh morgens unterrichten) dürfen auch verletzt werden. Hier geht es dann um Constraintoptimierung: Finde eine Lösung, die alle harten und möglichst viele Softconstraints erfüllt. Das geschieht in der Regel durch Hinzufügen immer weiterer Constraints, bis man keine Lösung mehr bekommt.

15 Von einfachen und schwierigen Constraintproblemen

Die Komplexität eines CSPs kann sehr einfach durch Multiplikation der Wertebereichsgrößen aller Variablen berechnet werden, genauso wie wir es von den Suchproblemen her kennen – dort haben wir das als Problemraumgröße bezeichnet. In der Praxis jedoch sind insbesondere auch die Constraints dafür verantwortlich, wie schwer ein CSP zu lösen ist. Beispiele:

- Ein CSP ohne Constraints ist trivial lösbar. Wir müssen einfach nur alle Variablen einmal belegen.
- Ein CSP mit wenigen Constraints ist „einfach“ lösbar. Wir belegen alle Variablen und müssen dabei nur 2 Constraints beachten.
- Mit jedem zusätzlichen Constraint wird ein CSP immer schwieriger lösbar. Denn jedes zusätzliche Constraint schränkt ja die Zahl der konsistenten Belegungen weiter ein. Z.B. bei Sudokus, wo im Normalfall nur eine einzige Lösung existiert. Hier müssen wir im Worst Case alle Variablenbelegungen durchführen (also den gesamten Problemraum aufspannen), um die Unlösbarkeit des CSPs zu erkennen bzw. um die einzige Lösung zu generieren.
- Irgendwann aber sinkt die Problemlösekomplexität wieder, wenn wir noch weitere Constraints einführen. Wenn nämlich ein CSP unlösbar ist, und wir weitere einschränkende Constraints hinzufügen, wird man in der Regel die Unlösbarkeit immer schneller feststellen können.

16 Backtrackingsuche verzahnt das Belegen und Testen

Backtracking ist eine Variante der Tiefensuche.

Die theoretische Komplexität eines CSPs hängt von der Zahl der Variablen und den Wertebereichsgrößen ab. Im Beispiel (Färbeproblem mit 5 Variablen und 3 Farben existieren insgesamt 3^5 vollständige Belegungen.

Backtracking ist eine Tiefensuchevariante: Nacheinander werden die Variablen konsistent belegt, bis wir auf der untersten Baumebene eine vollständige und konsistente Belegung für das CSP haben.

Konsistentes Belegen = Zuweisung eines Wertes, der kein Constraint zu bereits belegten Variablen verletzt.

Simulation: Wir belegen die Variablen in lexikographischer Reihenfolge und auch deren Werte.

Algorithmus (informell, ein konkreter folgt anschließend):

```
Repeat
        wähle unbelegte Variable xi
        belege xi konsistent  (quasi Expandierung; hier entstehen Backtrackingpunkte)
Until alle Variablen belegt
```

Start mit leerer Belegung.

1. a=rot. Die beiden gestrichelten Linien deuten alternative konsistente Belegungen an.
2. b=blau. Denn b=rot würde das Constraint b≠rot verletzen.
3. c=grün. Einzige Möglichkeit, denn alle anderen Belegungen würden entweder das Ungleichheitsconstraint zu a oder zu b verletzen.
4. Das konsistente Belegen der Variable d scheitert. Wir lösen Backtracking aus und gehen zwei Schritte im Suchbaum zurück (denn eine alternative konsistente Belegung für c existiert nicht).

17 Fortsetzung der Suche nach dem 1. Backtrackingschritt

Der inkonsistente Belegungspfad wird entfernt, und wir sind wieder im Zustand nach a=rot.

b=grün gefolgt von c=blau führt wieder in eine Sackgasse: Erneut kann d nicht konsistent belegt werden. Also noch einmal Backtracking auslösen.

Nun führt der Sprung zurück zur Wurzel, und a wird neu belegt...

18 Fortsetzung der Suche nach dem 2. Backtrackingschritt

Erneut wird im Tiefensuchestil der Sackgassenpfad entfernt und die nächste Belegung für a vorgenommen.

Nach a=blau folgen noch 4 weitere konsistente Belegungsschritte. Danach ist das CSP gelöst.

Es existieren nun noch 3 Backtrackpunkte (alternative Belegungen für c, b und a sind noch möglich). Das ist nur dann von Interesse, falls mehrere Lösungen gefunden werden sollen (was bei Graphenfärbung wenig Sinn ergibt)

19 Backtracking-Algorithmus mit Stack

1. alle Variablen sind als „nicht belegt“ markiert. Diese „leere Belegung“ wird initial auf den Stack gelegt.
2. Die Schleife bricht ab, wenn der Stack leer ist (dann endet Backtracking mit einer Fehlermeldung (CSP unerfüllbar)) oder wenn das oberste Element eine vollständige Belegung ist (jede Variable hat einen Wert, dann war die Suche erfolgreich und die Belegung wird zurückgeliefert (Schritt 7.).
3. In der Schleife wird das oberste Stackelement (welches eine unvollständige (aber konsistente) Belegung enthält) entfernt;
4. danach wird diese Belegung expandiert, indem eine unbelegte Variable ausgewählt wird und mit allen Werten ihrer Domain belegt wird. Die Wahl der als nächstes zu belegenden Variable kann durch eine Heuristik ermittelt werden (späteres Video).
5. Diese neuen Belegungen werden nun auf Erfüllbarkeit mit den bisherigen Belegungen gemäß der Constraints getestet. Alle inkonsistenten Belegungen werden verworfen.

6. Die restlichen Belegungen sind offensichtlich konsistent, sie werden auf den Stack gelegt. Und weiter geht es mit Schritt 2, wo die UNTIL-Bedingung geprüft wird.
7. Ende: Erfolgreich, wenn die Belegung vollständig ist (sie ist durch die vorherigen Constraintchecks automatisch auch konsistent) – in diesem Fall wird die Belegung zurückgeliefert. Fehler, falls der Stack leer ist – dann war das CSP nicht lösbar.

Dieser Algorithmus terminiert bei der ersten Lösung. Will man alle Lösungen generieren, müsste man jedes Mal, wenn man eine Lösung gefunden hat, diese vom Stack entfernen und dann weitersuchen.

20 Backtracking hat gute Eigenschaften für CSPs – bis auf die Zeitkomplexität

Backtracking = Tiefensuche, daher dieselben Leistungsmerkmale im Worst Case, hier aber mit den für CSPs typischen endlichen Bäumen.

n = Anzahl Variablen; b = maximale Domaingröße. Aufwand der Suche im Worst Case: b^n.

Weil die Zahl der Variablen und die finiten Domains endlich sind, ist Vollständigkeit sichergestellt. Optimalität spielt keine Rolle, irgend eine zulässige Lösung ist ok.

Dieser Aufwand wird für unlösbare CSPs von der Naiven Tiefensuche benötigt, die im Unterschied zum Backtracking das Generieren und Testen nicht verzahnt, sondern trennt: Zunächst werden alle Variablen belegt und danach die Constraints überprüft. Wir sprechen auch von Generate & Test. Diese Suche generiert also immer einen Lösungskandidaten und testet ihn dann.

Hier hat Tiefensuche also sowohl Vollständigkeits- als auch Optimalitätseigenschaft – aber nur für die einschränkende Bedingung endlicher Domains (sonst wäre der Verzweigungsgrad unendlich und es können nicht alle Lösungen generiert werden)!

Speicherkomplexität: Alle n Variablen mit ihren gesamten Wertebereichen (der Größe b) müssen gespeichert werden. Wenn das außerhalb des Zustands erfolgt, dann müssen während der Suche noch n Belegungen gespeichert werden. Das ergibt einen Aufwand von 2*n*b. Komplexer hingegen ist die Speicherung der Constraints: Wir benötigen (im Worst Case) für jedes Constraint eine Tabelle von b*b Werten (jeder Wert der einen Variable ist konsistent mit jedem Wert der anderen Variable - ein solches Constraint ist natürlich sinnlos, präziser wäre die Annahme: b * (b-1) Tabelleneinträge, aber diese Größe ist gemäß O-Notation gleichgroß wie b*b). Es existieren maximal n*n/2 Constraints (je ein Constraint zwischen jedem Paar von Variablen (Constraints sind ungerichtet)). Der Speicheraufwand für die Variablen „verschwindet“ in der O-Notation, weil er marginal ist im Vergleich zur Größenordnung von n*n*b*b der gespeicherten Constraints.

Die Speicherbilanz von Backtracking ist zufrieden stellend, nicht aber die Zeitkomplexität. Im Folgenden stellen wir domänenunabhängige Heuristiken vor, die auf eine optimale Belegungsreihenfolge abzielen. Das Laufzeitverhalten kann durch den Einsatz von Heuristiken in den meisten Fällen deutlich verbessert werden.

21 Quiz zur Belegungsreihenfolge bei Backtrackingsuche

(Überleitung zu Heuristiken)

Die Art der Constraints und die Wertebereiche seien unbekannt.

Welche Belegungsreihenfolge ist die beste? Annahme: Wir haben ein schwer lösbares CSP vorliegen.

Zur Beantwortung der Frage noch einmal die Backtrackingsuche vergegenwärtigen: Wir belegen eine Variable, und zwar konsistenz (zu den bereits belegten Variablen). Das Belegen und das Testen der Constraints sind im Backtracking also verzahnt.

22 Frühe Konsistenzchecks begrenzen den Suchbaum

Anders als bei Suchproblemen gehen wir nicht optimistisch daran möglichst schnell eine vollständige konsistente Belegung zu erreichen, sondern sind pessimistisch eingestellt und vermuten, dass die meisten Belegungen nicht zu Lösungen führen werden.

Was wir erreichen wollen, ist eine möglichst starke Beschneidung des Suchbaums. Die Maßnahmen, um den Baum möglichst stark zu beschneiden, fasst man auch unter dem Begriff "First Fail-Prinzip" zusammen.

Dahinter steht die Idee, die schwierigsten Entscheidungen als erstes zu treffen und nicht bis zum Schluss aufzusparen.

Die Belegungsreihenfolge der Variablen beeinflusst die Effizienz von Backtracking!

Idee: Die Belegungsreihenfolge der Variablen so wählen, dass möglichst viele Tests möglichst früh stattfinden. Im „konsistenten Belegen“ steckt der Verzweigungsgrad. Bei wenigen konsistenten Belegungen ist er geringer.

- Beispiel: Variablen: A,B,D:{1,2}; C:{2,3}. Constraints: A=C; B=C; C=D.
- A, B und D sind insofern voneinander unabhängig belegbar, weil keine Constraints zwischen diesen Variablen existieren. Inkonsistenzen werden erst erkannt, wenn C belegt wird. Die Belegungsreihenfolge (A,B,D,C) ist daher schlecht (Backtracking-Suchbaum links), denn alle Tests finden erst auf Baumebene 4 (bei der Belegung von C) statt. Im Suchbaum wird C sechsmal erfolglos belegt, erst ganz rechts kommt es zur Lösung A=2, B=2, D=2, C=2.
- In der Reihenfolge (A,C,D,B) findet der erste Test bereits auf Ebene 2 statt. Nach A=2 ist dann C nicht mehr konsistent belegbar und es wird Backtracking ausgelöst. Der resultierende Suchbaum ist deutlich kleiner als der linke.
- Andere Belegungsreihenfolgen sind genauso gut wie (A,C,D,B), beispielsweise C,A,B,D. Hauptsache, möglichst viele Constraints werden möglichst früh geprüft.A und B sind voneinander unabhängig belegbar. Inkonsistenzen werden erst erkannt, wenn C belegt wird. Die Belegungsreihenfolge (A,B,C,D) ist daher schlecht (Backtracking-Suchbaum links), denn alle Tests finden erst auf Baumebene 3 statt.

23 Kleine Wertebereiche machen das Scheitern wahrscheinlicher

Gemäß dem First-Fail-Prinzip #1 wäre a, b, c eine gute Belegungsreihenfolge.

Jedoch erkennt die Backtrackingsuche erst über 4-maliges Backtracking die Nichtlösbarkeit dieses CSPs. Dabei mussten 7 Belegungen durchgeführt werden.

Der Flaschenhals in diesem CSP ist der nur 2-stellige Wertebereich der Variable c. Beim Belegen von c hat der Suchbaum nur einen Verzweigungsgrad von 2, es ist also nur 1x Backtracking möglich.

Also zuerst c belegen, gefolgt von b und a: nach nur 4 Belegungen und nur 2 Backtrackingschritten steht die Unlösbarkeit fest.

Bemerkung: hier wurde eine andere Notation für das Belegen der Variablen verwendet: In jeder Zeile ist ein Belegungspfad notiert, bis zum Backtracking, das dann in der nächsten Zeile fortgesetzt wird.

24 Stark einschränkende Constraints zuerst auswerten

Starke Constraints zuerst: Die Gleichheitsconstraints e=f und f=a lassen ja nur einen Wert zu. Hingegen schließt ein Ungleichheitsconstraint nur einen Wert aus.

In der Arithmetik ist = stärker als >,< stärker als ≤,≥ stärker als ≠.

Eine gute Belegungsreihenfolge wäre: e, f, a, b, c, d. Ebenso gut möglich wäre f, a, e, d, c, b und so weiter.

Bemerkungen:

- Das all-different-Constraint (im Anhang) ist stärker als ≠-Constraints.
- Allgemein sind kleinere Relationen stärker als große. (Wenn wir die Constraints als Tabellen/Relationen formulieren)
- Höherstellige Constraints (A>B+C+D) sind im Allgemeinen auch stärker als zweistellige. Ein n-stelliges Constraint wird aber erst nach Belegung aller seiner n Variablen ausgewertet. Es entstehen dann also viele Backtrackingpunkte, weil die Auswertung erst spät erfolgt.

Abschließende Bemerkung: Es gibt keine Garantie, dass durch Anwendung der Heuristiken der Suchaufwand kleiner wird. Es lassen sich immer Gegenbeispiele konstruieren.

25 Chronologisches Backtracking leidet unter „dummen“ Rücksprüngen

Backtrackingsuche springt systematisch immer zum letzten Auswahlpunkt zurück (daher auch der Begriff *chronologisches Backtracking*)

Wir verwenden ein ähnliches CSP wie von Folie „Frühe Konsistenzchecks begrenzen den Suchbaum“, aus didaktischen Gründen aber mit einer anderen Belegungsreihenfolge.

1. A = 1
2. D = 1
3. B = 2

Nun scheitert die Belegung von C.

Jedoch führt der Rücksprung nach dem Scheitern von C=2 und von C=3 nur eine Stufe zurück (zur Belegung von B). Nach dem Backtrackingschritt erfolgt also

3. B = 3

und erneut kann C nicht konsistent belegt werden. Erst nach diesem erneuten Backtrackingschritt wird zurückgesprungen zur Variable D, die dann mit 2 belegt wird, was dann später erneut Backtracking auslöst, bis schlussendlich A=2 belegt wird).

Die Ursache für das Scheitern einer Belegung von C ist die Verletzung der Constraints C = A und C = D. Wir erkennen das in der Mitte der Folie, wo der Zustand nach A=1, D=1, B=2 dargestellt ist: C hat nur Werte in seiner Domain, die sowohl mit A als auch mit B in Konflikt stehen.

Hier wäre es im Sinne einer schnelleren Lösungsfindung effektiver, wenn nach A=1, D=1, B=2 und dem Scheitern der Belegung von C gleich an die richtige Stelle im Suchbaum zurückgesprungen wird: nämlich zur Belegung von A!

So genanntes "intelligentes Backtracking" analysiert die Ursachen des Scheiterns. Danach wird zum höchsten Punkt im Baum zurückgesprungen, der ursächlich für den Konflikt ist. Im Beispiel also zum Belegen von A. Diese Ursachenanalyse kostet Rechenzeit, sie ist aber in den meisten Fällen (d.h., bei hinreichend komplexen und schwer zu lösenden CSPS) irrelevant wegen der großen Einsparungen im exponentiell wachsenden Suchbaum.

Mit dem nachfolgend vorgestellten Forward Checking bekommen wir "intelligentes Backtracking" geschenkt (und darüber hinaus benötigen wir auch keine Ursachenanalyse).

26 Agenda Lokale Constraintpropagierung / partial look ahead

Lösung von CSPs durch Propagierung:

Das Backtracking- Suchverfahren verwendet die Constraints „passiv“, bzw. a posteriori: Ein Constraint wird erst dann geprüft, wenn alle seine Variablen belegt sind. Constraintpropagierung geht einen anderen Weg: Constraints werden aktiv genutzt, indem inkonsistente Werte aus den Domains von Variablen entfernt werden, die noch nicht belegt sind. Dadurch wird der Suchraum a priori eingeschränkt (verkleinert ohne Suche), sodass deutlich weniger Knoten expandiert werden müssen, um zu einer Lösung zu gelangen. Denn durch das Entfernen „offensichtlich inkonsistenter“ Werte aus den Domains wird der Suchbaum in seiner Breite reduziert.

Die möglichst frühzeitige und umfassende Auswertung von Constraints ist das Thema dieses Abschnitts. Wir werden sehen, dass Forward Checking ein wenig aufwändiges Verfahren ist, das effizienter ist als das bestmögliche „intelligente“ Backtracking.

27 Lösen von Sudokus mittels Forward Checking (und Backtracking)

(Sudoku-Beispiel vertieft in Screencast 04A06 Forward Checking)

Zustand = Wertebereiche aller Variablen. Zum Vergleich: Bei einer reinen Backtrackingsuche müssen im Zustand nur die Belegungen gespeichert werden.

Wir belegen eine Variable (z.B. unten links v(a,1)=4) und werten danach alle Constraints zu noch nicht belegten Variablen aus. (v(a,2), v(a,4), ..., aber nicht v(a,3) weil diese Variable ja bereits belegt ist).

Die Auswertung der Constraints führt dann zu Wertebereichseinschränkungen in den „benachbarten" Variablen (das sind alle diejenigen Variablen, die mit der gerade belegten Variable über ein Constraint verknüpft sind (die also im Constraintgraph mit der gerade belegten Variable verbunden sind)): Weil alle Constraints Unbleichheitsconstraints sind, bedeutet dies, dass wir den bei der Belegung verwendeten Wert aus den Domains der benachbarten unbelegten Variablen entfernen. Im Sudoku-Beispiel wird also die „4" in den Feldern der ersten Spalte, der unteren Reihe und dem Quadranten gestrichen – bzw. im Sprachgebrauch der CSPs: „wird der Wert 4 aus den Domains der noch unbelegten Variablen ... entfernt). Im Beispiel sind davon betroffen die vier Variablen v(a,2), v(c,2), v(c,3), v(e,1).

Dieser Propagierungsschritt "in die Zukunft" wird Forward Checking (abgekürzt: FC) genannt.

Mit Forward Checking verkleinern wir Wertebereiche und verringern damit den Verzweigungsgrad des Suchbaums.

Constraints zu bereits belegten Variablen wurden ja bereits zuvor gecheckt (bei ihrer Belegung), sie müssen nicht erneut einer Prüfung unterzogen werden.

Wir erkennen, dass die FC-Propagierung zum Lösen eines CSPs nicht ausreicht. Wir müssen weiterhin eine Suche durchführen. Für die weitere Suche haben wir aber einen deutlich in der Breite reduzierten Baum.

Dazu verwenden wir eine angepasste Backtrackingsuche...

28 Backtrackingsuche in Verbindung mit Forward Checking begrenzt den Suchraum

Beispiel Graphenfärbung mit 5 Knoten, 8 Constraints und 3 Farben (der initiale Wertebereich der Variable b besteht nur aus 2 Farben):

$$D_x = \{ r, b, g \}\, \forall x \in \{a, c, d, e\},\ D_b = \{g, b\}$$

$$a \neq c,\ a \neq d,\ a \neq e,\ b \neq c,\ b \neq d,\ b \neq e,\ d \neq e$$

Die Folie zeigt den Backtracking&Forward Checking-Suchbaum, von links nach rechts aufgespannt. Jedes grau gefärbte Feld stellt einen Zustand dar mit Domainvariablen. Belegte Variablen sind für eine bessere Übersicht als gefärbte Knoten dargestellt.

1. (ganz links) Start mit leerer Belegung, alle Variablen haben ihre initialen Domains. Insgesamt existieren 3^4 *2 = 162 vollständige Belegungen.
2. (nach rechts oben) Belegung a=r. Auswertung der Constraints a≠c, a≠d, a≠e (rot gefärbte Kanten), dadurch Reduzierung der Wertebereiche von c, d und e: es wird der belegte Wert „r" aus ihren Domains entfernt. Nach diesem Schritt ist der Suchraum auf 2^4=16 vollständige Belegungen verkleinert.
3. (nach oben rechts) Belegung b=g. Auswertung der 3 rot dargestellten Constraints. Die zu b benachbarten Variablen haben nun einstellige Wertebereiche, es existiert in dieser Situation also nur noch 1 vollständige Belegung.
4. (nach rechts) Belegung c=b und Auswertung des Constraints zwischen c und d. Dadurch erhält d einen leeren Wertebereich. Wir haben eine Sackgasse erreicht, denn ein leerer Wertebereich bedeutet ja, dass die betroffene Variable nicht mehr belegt werden kann. Wir führen also einen Backtrackingschritt aus. Auf der vorherigen Ebene (Schritt 3.) existiert keine alternative Belegung für c. Also wird eine weitere Ebene zurückgesprungen…
5. (in Situation nach 2. Nach rechts unten) Belegung b=b und Auswertung der 3 Constraints (analog dem Schritt 3.). Es entsteht eine mit Schritt 3. ähnliche Situation.
6. (nach rechts) Belegung c=b und erneut Auslösen von Backtracking, weil der Wertebereich der Variablen d leer wird. Nun erfolgt der Rücksprung bis zur Wurzel des Suchbaums (also die 0. Ebene) …
7. (ganz links nach unten rechts gehen) a=g und Auswertung der 3 Constraints, die a enthalten.
8. b=g und Auswertung der 3 b-Constraints. Nun besitzen die 3 noch unbelegten Variable jeweils 2-stellige Domains.
9. c=r und die Forward-Checking-Propagierung schränkt den Wertebereich der Variablen d auf {b} ein.

10. d=b und e erhält eine einstellige Domain. Nun sind alle Constraints im CSP ausgewertet. Die letzte Variable e kann ohne weitere Tests einfach belegt werden
11. e=r und wir haben eine vollständige und konsistente Belegung.

29 Forward Checking realisiert ein „partial look ahead“

Gegeben ein CSP mit 10 Variablen und 11 Constraints.

4 Variablen wurden belegt, die schwarz gefärbten v1 bis v3 in der Vergangenheit, die graue v4 im aktuellen Belegungsschritt der Backtrackingsuche. Die weißen, noch nicht belegten Variablen liegen in der Zukunft – sie werden später belegt.

Betrachte den Zustand „Gegenwart“, wo die grau gefärbte Variable belegt wird:

- Backtracking (ohne FC) überprüft nur das Constraint mit der bereits früher belegten Variable v3. Dieses Constraint liegt „in der Vergangenheit“, weshalb man auch von „look back“ spricht. Die Constraints zwischen den Knoten v1-v2 und v2-v3 wurden bei einer Backtrackingsuche bereits bei den vorherigen Belegungen von v2 bzw. v3 untersucht; alle anderen Constraints wurden noch nicht geprüft.
- Forward Checking hingegen realisiert ein einfaches „look ahead“, wobei alle Constraints zwischen der aktuell belegten Variable v4 und den noch nicht belegten Variablen v6 und v7 ausgewertet werden. Die Reichweite ist also hier „2 Variablen in die Zukunft“). Bei den früheren Belegungsschritten wurde dasselbe getan: Mit der Belegung von v1 wurde das Constraint mit v2 geprüft, einen Belegungsschritt später die 3 Constraints mit v3, v5 und v6, und bei der Belegung von v3 die beiden Constraints mit v4 und v8. Im aktuellen Zustand wurden nur die Constraints zwischen den noch unbelegten Variablen (v5-v9, v7-v9, v9-v10) noch nicht untersucht.

30 Integration von Forward Checking in die Backtrackingsuche

1-3 und 6 realisieren die Backtrackingsuche durch sukzessives Belegen von Variablen. Wichtiger Unterschied: BT&FC[1] wählt irgendeinen Wert aus *ohne Prüfung von Constraints*. Denn der Blick in die Vergangenheit zu bereits belegten Variablen ist unnötig, weil die Constraintprüfung ja nach vorne in die Zukunft stattfindet. Und alle früher belegten Variablen, die ein Constraint mit der aktuellen Variable x haben, sind daher bereits ausgewertet worden, sodass x nur noch (in Bezug auf vergangene Belegungen) konsistente Werte enthält.

(1) REPEAT
(2) Wähle unbelegte Variable x
(3) belege x mit Wert $v \epsilon D_x$
(4) Entferne für jede unbelegte Variable y, die in einem Constraint $c(x, y)$ vorkommt, alle mit v inkonsistenten Werte
(5) Falls $D_y = \{\}$ existiert, löse Backtracking aus
(6) UNTIL alle Variablen belegt

(4) und (5) setzen FC um: Wir betrachten nacheinander alle (zweistelligen) Constraints, die die Variable x und eine noch unbelegte Variable y enthalten. Dazu führen wir für jeden Wert in Dy den Constraintcheck mit v durch. Alle Werte, die diesen Check erfüllen, bleiben in Dy erhalten, die inkonsistenten Werte werden entfernt.

Beispiele:

- sei x=3 die Belegung, Da=Db=Dc={1...5} und c(x,a) = "x=a", c(x,b) = "x<b", c(x,c) = "x≠c"
- dann lauten die durch Forward Checking veränderten Wertebereiche: D´a={3}, D´b={4,5}, D´c={1,2,4,5}

Eine reine BT-Suche würde diese Tests erst beim Belegen durchführen. Der Aufwand um alle konsistenten Belegungen zu generieren ist derselbe, nur dass er beim FC zeitlich vorgezogen wird.

[1] Wir verwenden die Abkürzungen BT für Backtracking und FC für Forward Checking.

(5) prüft, ob durch diese Propagierung irgendein Wertebereich leer wurde. In diesem Fall liegt eine Inkonsistenz vor und es wird Backtracking ausgelöst. Der Backtrackingschritt führt zurück zu Zeile (3), die aktuelle Variable wird mit einem anderen Wert belegt. Falls kein weiterer Wert mehr existiert, wird eine Ebene zur vorherigen Variable zurückgesprungen.

31 Forward Checking hat quadratischen Zeitaufwand

Aufwandsberechnung: Bei n Variablen mit $|D_{xi}| = b$ (n und b für Tiefe und branching factor im Baum) existieren maximal n*n/2 Constraints. Forward Checking betrachtet aber immer nur alle Constraints, die die gerade instanziierte Variable enthalten - also müssen maximal n-1 Constraints für eine instanziierte Variable gecheckt werden. Jede Constraintüberprüfung besteht aus b Vergleichen (mit dem instanziierten Wert). Der Aufwand beträgt also
$O(b * n)$.

Quadratischer Aufwand ist sehr günstig im Vergleich zum exponentiellen Aufwand der Backtrackingsuche. D.h. der Mehraufwand durch das look ahead lohnt sich in komplexen CSPs eigentlich immer.

Der Speicheraufwand ist von geringerem Interesse, weil der Stack der Backtracking-Tiefensuche ja immer nur einen Pfad beinhaltet. Der gesamte Zustand (alle Variablen mit ihren aktuellen Domains muss im Zustand verwaltet werden, die Constraints als „Atemporale“ können auch außerhalb des Suchbaums (quasi statische Konstanten) gespeichert werden.

FC ist genauso gut wie das intelligente BT, stellt die Analyse aber im Voraus an, sodass weniger Variablen belegt werden. Die Berechnungen sind dafür komplexer als das Prüfen eines vollständig instanziierten Constraints.

32 Aggressives Forward Checking propagiert einstellige Wertebereiche bis zum Stillstand

FC propagiert die Auswirkungen einstelliger Domains an die "benachbarten" Variablen. Durch diesen Propagierungsschritt können weitere einstellige Domains entstehen. Aggressives FC setzt die FC-Propagierung für alle solche einstelligen Variablen fort bis zum Stillstand, oder bis ein leerer Wertebereich auftritt.

Links: Das CSP im Initialzustand, rechts: nach dem aggressiven Forward Checking.

Unten: die Entwicklung der internen Datenstrukturen im Rahmen einer Backtrackingsuche mit aggressivem Forward Checking. Auf dem Weg zur Lösung muss jedes Constraint nur 1x ausgewertet werden (nur nach Backtrackingschritten erneut). Wir speichern die Variablen mit einstelligem Wertebereich, auf die FC angewendet werden muss, sowie die noch offenen (d.h. nicht ausgewerteten) Constraints. Außerdem für jede Variable deren Wertebereich.

1. Belegung a=1 im Backtrackingalgorithmus. a ist damit die FC-Variable, deren Wert propagiert werden muss...
2. Das offene Constraint a>b wird ausgewertet, Db={1} auf 1 Wert eingeschränkt. An dieser Variable wird das aggressive FC fortgeführt: b wird in die Liste der FC-Variablen aufgenommen.
3. b tritt in 3 offenen Constraints (b≠c, b<d, b≠e) auf. Diese werden nacheinander ausgewertet: Zuerst wird der Wert 1 aus Dc entfernt, wodurch c einstellig wird.
4. Danach wird das Constraint b<d geprüft. d=1 verletzt das Constraint, d=2 erfüllt das Constraint, und somit ist nun auch Dd einstellig.
5. Durch Auswertung des Constraints b≠e wird der Wert 1 aus De entfernt. Nun ist die FC-Propagierung für b abgeschlossen.
6. In der Tabelle (Zeile 4) sind nun die einstelligen Variablen c und d aufgeführt, für die das aggressive FC fortgeführt wird. Von den anfangs 7 Constraints sind nun bereits 4 ausgewertet.
7. Wir entfernen c aus der FC-Variablenliste und führen Forward Checking für das einzige betroffene Constraint c=d aus. Die Auswertung ergibt keine Änderung im Wertebereich von d.
8. Nun liegt nur noch d in der FC-Variablenliste. d hat ein offenes Constraint zu e: d≤e. Dessen Auswertung führt zu keiner Wertebereichseinschränkung.
9. Die Liste der FC-Variablen ist leer und die aggressive FC-Propagierung endet.

Von den ursprünglich 7 offenen Constraints wurden nach Belegung von a also bereits 6 endgültig ausgewertet. Nach nur einem Belegungsschritt (a=2) enthält das CSP nun nur noch 6 vollständige Belegungen (der Suchbaum hat einen Verzweigungsgrad von 1 für b, c und d, von 2 für e und von 3 für f).

Für den Fall, dass im Rahmen der FC-Propagierung ein Wertebereich leer wird, endet die Propagierung sofort und es wird Backtracking eingeleitet.

33 Die Gradheuristik beschleunigt die Backtracking+Forward Checking-Suche

Die für chronologisches Backtracking eingeführten Heuristiken (Folie #22 ff) funktionieren auch - und noch viel besser – mit BT+FC.

Recap First Fail-Heuristik: Annahme ist ein schwierig zu lösendes CSP (wohlmöglich ein unlösbares). Dabei sollen die kritischen Entscheidungen (= schwierig zu belegende Variablen) als Erstes getroffen werden. Somit wird Backtracking möglichst früh (und nicht erst unnötig spät) ausgelöst, was den Verzweigungsgrad des Suchbaums bereits frühzeitig verringert.

Backtracking+Forward Checking+Gradheuristik (BT+FC+Gradheuristik): Gezählt wird für jede Variable die Zahl der Constraints *mit noch nicht belegten Variablen*. (d.h. die offenen (noch nicht ausgewerteten) Constraints).

Die Idee dabei: Je mehr offene Constraints eine Variable hat, desto weit reichender ist die Propagierung nach dem Belegen dieser Variable. Constraints zu den bereits belegten Variablen mitzuzählen ergibt keinen Sinn, da diese ja nicht erneut geprüft werden.

1. Beispiel links: In den Knoten sind die Gradwerte vermerkt: x und y haben mit je 4 ausgehenden Kanten die meisten Constraints. Die Variable z, die ohne Constraint ist, hat konsequenterweise den kleinsten möglichen Grad 0 und wird daher zuletzt belegt. Ihre Belegung trägt nichts zum First Fail Prinzip bei ("belege so, dass eine Constraintverletzung am wahrscheinlichsten ist").
2. Beispiel Mitte: Nach dem Belegen von x verringert sich der Grad der mit x in Constraintbeziehung stehenden Variablen um -1. Nun hat y den höchsten Grad von 4.
3. Beispiel rechts: Nach Belegung der Variablen x und y besitzen alle Variablen einen Grad von 0. Das bedeutet, dass sämtliche Constraints bereits ausgewertet wurden. Die noch nicht belegten Variablen können also mit beliebigen Werten ihrer aktuellen Domains belegt werden! Mit anderen Worten: bereits nach 2 Variablenbelegungen findet kein einziger Backtrackingschritt mehr statt!

Die Gradheuristik eignet sich besonders gut in Verbindung mit Forward Checking: Wir wissen ja bereits, dass man bei BT+FC jedes Constraint nur exakt einmal „anfassen“ muss. Und je früher Constraints ausgewertet werden, desto größer die zu erwartende Suchbaumbeschneidung.

Bemerkungen:

- Vergleich zur Gradheuristik in der reinen Backtrackingsuche (Folie #22, First-Fail-Prinzip #1): Dort errechnet sich der Grad nach der Zahl der Constraints zu bereits belegten Variablen. Der Unterschied liegt in der rückwärtsgerichteten Backtrackingsuche versus dem vorwärtsgerichteten Forward Checking.
- Die Belegungsreihenfolge der Variablen gemäß Gradheuristik ist statisch, sie kann vorab berechnet werden.
- Variablen wie z, die unconstrained sind, werden natürlich zuletzt belegt. Denn durch ihre Belegung kann in keinem Fall Backtracking ausgelöst werden.

34 Minimum Remaining Values-Heuristik (MRV) minimiert den Verzweigungsgrad des Suchbaums

Backtracking+Forward Checking+Minimum Remaining Values (BT+FC+MRV)

Motivation: Je kleiner der Wertebereich, desto geringer der Verzweigungsgrad. Also weniger Rücksprungmöglichkeiten und damit stärkere Beschneidung des Suchbaums (und das entspricht wiederum der First-Fail-Idee). Wir betrachten den dargestellten Initialzustand:

- a hat eine Wertebereichsgröße 2 (notiert als $|D_a| = 2$). Wenn die erste Belegung zu einer Constraintverletzung führt, dann ist der Suchbaum anschließend um 50% kleiner.

- $|D_c| = 4$: Mit jeder Belegung für c generieren wir nachfolgend immer nur 25% des Suchbaums. D.h. das Scheitern einer Belegung für c führt zu einer Ersparnis von 25% (sofern das Scheitern sofort erkannt wird).
- Für eine Variable $|D_x| = 1$ ist das noch evidenter: Scheitert das Belegen dieser Variable, dann ist das CSP endgültig als unlösbar erkannt.

Simulation BT+FC+MRV mit Gradheuristik als Tiebreaker: (Spalte 1 enthält den aktuellen Belegungsschritt, Spalte 2 die jeweilige MRV-Variable(n) nach diesem Belegungsschritt, die anderen Spalten die Wertebereiche nach dem Forward Checking-Schritt, der auf die Belegung erfolgte.

1. (leere Belegung) MRV ist a mit einer Wertebereichsgröße von 2 (notiert als: a(2)).
2. Wir belegen a=1 (wir wählen immer den ersten möglichen, hier also: kleinsten, Wert) und werten das Constraint zwischen a und b aus. Die Variable b erhält einen leeren Wertebereich. Daher Backtracking auslösen...
3. Wir belegen (im Initialzustand) a=2. Nun verkleinert sich D_b auf {2}. In dieser Situation ist b mit nur einem Wert (notiert als b(1)) die Minimum Remaining Values Variable.
4. Nach Belegung b=2 ergibt sich durch die FC-Propagierung D_c= {1,3,4}. D_d und D_e bleiben unverändert. Nun haben wir eine Tiebreaksituation: alle offenen Variablen haben einen gleich großen Wertebereich. Wir verwenden die bereits bekannte Gradheuristik als Tiebreaker. d mit noch 2 offenen Constraints ist gemäß Gradheuristik die First-Fail-Variable.
5. d=1 belegen und die beiden offenen Constraints zu c und e auswerten. Effekt: D_c= {3,4}, D_e= {1}. Nun sind bereits sämtliche Constraints ausgewertet und wir wissen, dass für das CSP Lösungen existieren. Die nachfolgenden Schritte sind nun trivial: Zunächst ist e MRV.
6. e=3 und keine Constraintauswertung. Nun ist die letzte noch unbelegte Variable c(2) MRV.
7. c=3 ohne Constraintauswertung. Alle Variablen belegt, CSP gelöst!

Bemerkungen:

- Auch Variablen mit einstelligem Wertebereich müssen noch belegt werden, wie in den Schritten 4. und 6. Denn diese können ja noch offene Constraint zu noch nicht belegten Variablen haben.
- Die Belegungsreihenfolge kann nicht a priori bestimmt werden, denn die Wertebereichsgrößen ändern sich im Laufe der Suche dynamisch.

Zusammenfassung:

- Auch die First-Fail-Heuristik "Variablen mit stark einschränkenden Constraints zuerst belegen“ (Folie #34) ist sinnvoll einsetzbar.
- Die Kombination von Heuristiken ist sinnvoll, um Tiebreaksituationen aufzulösen.
- Die Gradheuristik kann a priori bestimmt werden. Hingegen muss die Belegungsreihenfolge mit MRV dynamisch ermittelt werden.
- Welche Kombination sinnvoll ist, ergibt sich durch Studium des CSPs: Bei gleichgroßer Vernetzung aber unterschiedlichen Wertebereichen ist MRV besser geeignet als die Gradheuristik. Dies ist beispielsweise der Fall im Sudoku, nachdem für alle vorbelegten Felder die FC-Propagierung durchgeführt wurde. Umgekehrt ist die Gradheuristik bevorzugt anzuwenden, wenn die Wertebereiche ähnlich groß sind, aber die Vernetzung sehr unterschiedlich.
- Alle diese Heuristiken sind domänenunabhängig (und dem entsprechend generell einsetzbar, u.U. aber auch mit nur geringem Erfolg).

35 Heuristiken steigern die Effizienz von Backtracking mit Forward Checking

(Zusammenfassung)

In der Literatur wird noch eine weitere Heuristik erwähnt: Least Constraining Value (LCV): Wenn man sich schon für eine schwierig zu belegende Variable entschieden hat, dann belegen wir sie mit dem wahrscheinlichsten Wert, der (voraussichtlich) keine Constraintverletzung auslöst (also den “least constraining value”). Heuristiken steigern die Effizienz von Backtracking mit Forward Checking

Das Problem mit dieser Heuristik ist, dass dieser Wert nicht einfach zu finden ist – es bedarf einer Analyse der Constraints (mithin eine Art Forward Checking), was wir aber eigentlich vermeiden möchten, denn eine Heuristik soll ja insbesondere schnell berechenbar sein.

Aber in manchen Fällen ist die Anwendung von LCV einfach: Bei Zahlenwertebereichen und einem Constraint A>B erfüllt bei der Belegung von A der größte Wert die LCV-Heuristik. (Schwieriger wird es natürlich, wenn A noch mit einer anderen Variablen in "<"-Beziehung steht.)

36 Agenda: Lokale Konsistenz in CSPs

Forward Checking blickt in die unmittelbare Zukunft und realisiert somit ein „partial look ahead". Man kann die Constraintpropagierung aber auch weiterführen, indem Constraints auch ohne Variablenbelegung ausgewertet werden und zwar global (alle Constraints betreffend). Hier sprechen wir von einem full look ahead. Es handelt sich um Mechanismen, die ein CSP in einen Zustand der *Konsistenz* überführen. Deren Wirkung auf das Beschneiden des Suchbaums ist noch stärker als die FC-Propagierung.

Im Folgenden werden wir unterschiedliche Grade der Konsistenz betrachten. Schwerpunkt unserer Betrachtungen sind aus Komplexitätsgründen 1- und 2-Konsistenz. 1-Konsistenz wird auch als Knotenkonsistenz (node consistency), 2-Konsistenz auch als Kantenkonsistenz (arc consistency) bezeichnet.

37 Constraintpropagierung *vor* einer Belegung kann die Effizienz der Suche erheblich steigern

Im Beispiel zeigt sich BT+FC als wenig hilfreich, denn viele Variablenbelegungen (A=1, ... A=98) führen direkt zu Constraintverletzungen (und lösen Backtracking aus), bevor mit A=99 eine erste konsistente Belegung gefunden wird. (Die gestrichelten Linien in den Grafiken deuten an, dass dieses Constraint nur ein Ausschnitt eines größeren CSP ist.)

Constraintpropagierung kann aber auch bereits *vor* der Belegung der ersten Variablen stattfinden: Das Gleichheitsconstraint kann ausgewertet werden zu $D_A = D_B = \{99, 100\}$. Ergebnis ist eine erhebliche Suchraumverkleinerung. Der Effekt ist ein leichter zu lösendes CSP mit kleineren Wertebereichen und somit weniger Suche!

Analysiert man das Constraint, ohne irgendwelche Variablen zu belegen, kommt man schnell zum Schluss, dass konsistente Wertebereiche (unter Beachtung dieses 2-stelligen Constraints zwischen A und B) aus 2 Werten für A (99, 100) und 2 Werten für B (99, 100) besteht.

Diese Art der Constraintauswertung (vor dem Belegen einer Variable) ist eine andere Art der Constraintpropagierung als das bereits bekannte Forward Checking, das ja eine Variablen mit einstelliger Domain voraussetzt.

38 In einem 1-konsistenten CSP sind alle einstelligen Constraints erfüllt

Jede Variable hat in ihrer Domain nur solche Werte, die direkt kein Constraint verletzen. D.h., alle Werte in der Domain erfüllen das 1-stellige Constraint dieser Variable (sofern ein solches Constraint existiert, "mehrere" 1-stelligen Constraints einer Variable sind wie ein komplexeres einstelliges Constraint zu behandeln).

Mehrstellige Constraints spielen keine Rolle bei 1-Konsistenz.

Einstellige Constraints tauchen z.B. in Sudokus bei den vorbelegten Feldervariablen oder im JSS für Endtermine auf.

Wir erweitern den Begriff der Konsistenz nun auf das gesamte CSP: Ein CSP ist 1-konsistent, wenn alle Variablen 1-konsistent sind.

Wir stellen 1-Konsistenz her, indem wir die inkonsistenten Werte aus den Domains ALLER Variablen entfernen. Diese Berechnungen sind billig, wir müssen je Constraint einmal über alle Werte iterieren.

Beispiel links:

- Dieses CSP ist nicht 1-Konsistent, denn z.B. $1 \in D_a$ verletzt das Constraint a>4.
- Durch Auswertung der beiden 1-stelligen Constraints kann das CSP in einen 1-konsistenten Zustand überführt werden (die Variable b behält ihren ursprünglichen Wertebereich).
- Im 1-konsistenten Zustand existieren noch 2x3=6 vollständige Belegungen, im Originalzustand 6x3=18.

Beispiel rechts:

- Das Constraint c=1 wird für jeden Wert D_c verletzt. Das CSP ist also nicht 1-konsistent und kann auch nicht in einen 1-konsistenten Zustand überführt werden.
- Die Auswertung des Constraints ergibt einen leeren Wertebereich für c. Dies ist gleichbedeutend mit der Unlösbarkeit.

Bemerkungen:

- Einfachster Grad der Konsistenz. Führt durch Auswertungen bereits zur Suchraumverkleinerung. Kann gegebenenfalls bereits ohne eine einzige Variablenbelegung die Unlösbarkeit aufzeigen.
- hier stehen die Namen der Variablen in den Knoten
- Der englische Begriff ist "node consistency" (Knotenkonsistenz, weil ja nur einzelne Variablen=Knoten untersucht werden).
- Genauso wie beim Forward Checking brauchen die 1-stelligen Constraints, nachdem sie ausgewertet wurden, nicht mehr betrachtet werden.

39 Zweikonsistenz betrifft nur binäre Constraints

Der nächst höhere Grad ist 2-Konsistenz Hierfür betrachten wir nur die binären Constraints.

Definition über gerichtete Kanten (englischer Begriff ist "arc consistency" = Kantenkonsistenz): Jedes 2-stellige Constraint $c(x_i, x_j)$ besteht dann aus 2 Kanten (x_i, x_j) und (x_j, x_i) (Hin- und Rückrichtung). Ein Constraint ist ja immer ungerichtet, muss daher aus beiden Richtungen betrachtet werden.

Eine Kante ist 2-konsistent, wenn für jeden Wert der Quellvariable x_i mindestens ein Wert in der Zielvariable x_j existiert, der das Constraint erfüllt. Trifft diese Eigenschaft für alle Kanten zu, ist das CSP 2-konsistent.

Zur Illustration das CSP von der letzten Folie. Wir stellen 2-Konsistenz her:

- Zuerst betrachten wir die Kante (a,b) (a=Quelle, b=Ziel). Für a=1 existiert kein Wert in Db, der das Constraint erfüllt. Dito für a=2, 5 und 6. Für die anderen 3 Werte hingegen ist die 2-Konsistenzbedingung erfüllt.
- Nun noch die andere Richtung betrachten: (b,a). Hier sind alle Werte in b konsistent mit einem Wert in a. Also keine Werteelimination in dieser Richtung.
- Mit diesen Wertebereichen ist das CSP 2-konsistent.

Dieser Zustand mit $D_a=D_b=\{2,3,4\}$ ist aber nicht 1-konsistent, wie Sie sich überzeugen können, wenn Sie zur letzten Folie zurückspringen.

Die Kombination von 1- und 2-Konsistent wird starke 2-Konsistenz genannt: Wir stellen zunächst 1-Konsistenz her und danach 2-Konsistenz. Das vorliegende CSP ist nicht in einen stark 2-konsistenten Zustand überführbar. Ohne eine Variablenbelegung, alleine durch Konsistenzmechanismen konnte in diesem Fall die Unlösbarkeit erkannt werden.

1-Konsistenz und 2-Konsistenz sind offensichtlich notwendige Bedingungen für die Lösbarkeit eines CSPs. Aber sind sie auch hinreichend? (Antwort auf übernächster Folie)

40 Zweikonsistenz betrifft alle Variablenpaare

Andere Definition der Zweikonsistenz: In einem 2-konsistenten CSP ist jeder Wert einer Variablen konsistent zu jedem Wert einer beliebigen anderen Variable (d.h., ich kann 2 Variablen beliebig belegen und kein Constraint wird dadurch verletzt).

Beispiel links:

- Die 4 Kanten zwischen a und b und zwischen b und c sind 2-konsistent, aber nicht die beiden Kanten zwischen a und c.
- Stellen wir nun 2-Konsistenz für diese beiden Kanten her, ergibt sich das Bild rechts.

Fortführung des Beispiels rechts:

- Nun sind die Kanten zwischen a und c zweikonsistent und außerdem weiterhin die Kante von c nach b. Denn ich kann c=5 belegen und dann ist b=4 immer noch konsistent belegbar.
- Aber nicht mehr 2-konsistent sind die Kanten $\{(a,b),\ (b,a),\ (b,c)\}$. Beispielsweise nach b=4 kann c nicht mehr konsistent belegt werden, oder a=5 verletzt das Constraint zu b, denn kein Wert in b ist kleiner als 5.

Wichtige Erkenntnis: Einmaliges Herstellen 2-konsistenter Kanten reicht offensichtlich nicht aus! Den Algorithmus zum Herstellen von 2-Konsistenz lernen wir im nächsten Abschnitt kennen.

41 Höhere Grade der Konsistenz

Das dargestellte Graphenfärbeproblem mit 3 vollständig vernetzten Knoten und 2-stelligen Wertebereichen ist 2-konsistent aber offensichtlich unlösbar.

Also ist Zweikonsistenz keine hinreichende Bedingung für die Lösbarkeit eines CSPs.

Erweiterung auf k-Konsistenz:

- Für jede konsistente Belegung verschiedener $k-1$ Variablen ist jede k. Variable konsistent belegbar.
- Diese Definition passt auch zum bereits behandelten Fall von k=2.
- Eine konsistente Belegung von k-1 Variablen kann z.B. durch eine Backtrackingsuche generiert werden. Haben wir eine solche generiert, dann ist bei k-Konsistenz sichergestellt, dass wir eine beliebige andere Variable immer noch konsistent belegen können.

Das Beispiel links ist nicht 3-konsistent, denn z.B. nach a=1 und b=2 (konsistente Belegung von 2 Variablen, die das Constraint zwischen a und b nicht verletzt) ist c nicht mehr konsistent belegbar.

Das Beispiel rechts stammt von Freuder (1982). Es liegt keine 2-Konsistenz vor, denn y=2 verletzt sogar beide Constraints. Das CSP ist aber 3-konsistent: Denn die konsistente Belegung der 2 Variablen x und y (x=1, y=2) erlaubt eine konsistente Belegung für z (z=1). Und desgleichen für alle anderen konsistenten Belegungen von 2 Variablen. Weil das CSP nur 3 Variablen enthält, ist 3-Konsistenz gleichbedeutend mit n-Konsistenz und daher wissen wir, dass das CSP lösbar ist.

Allgemein gilt: bei n Variablen ist ein n-konsistentes CSP garantiert lösbar. Problem: Herstellen von n-Konsistenz kostet Zeit (b^n, wenn b die Größe des größten Wertebereichs ist, also genau die Zeit, die der Aufwandsklasse des CSPs entspricht).

Starke (strenge[2]) 3-Konsistenz schließt 1- und 2-Konsistenz mit ein und stellt dann einen höheren Grad an Konsistenz her.

Ein stark n-konsistentes CSP (das also jeden Grad 1...n an Konsistenz beinhaltet) kann mittels Backtracking ohne einen einzigen Backtrackingschritt gelöst werden.

E.C. Freuder: A Sufficient Condition for Backtrack-Free Search. In: Journal of the ACM (JACM) 29, 1982.

42 AC-3-Algorithmus zur 2-Konsistenz arbeitet auf gerichteten Zweigen

Idee: Alle Zweige werden ausgewertet. Wenn eine Variable dadurch eine Wertebereichseinschränkung erfährt, müssen die Zweige in diese Variable erneut betrachtet werden. Quasi eine Weiterpropagierung von Zweigen bis zum Stillstand oder zur Erkennung einer Inkonsistenz.

1. Aufruf mit V=Variablen, D=Domains, C=Constraints
2. Für jedes 2-stellige Constraint werden 2 Kanten in die Queue gelegt.
3. Die Menge der Kanten in der Queue wird nachfolgend abgearbeitet.
4. Eine Kante wird aus Q entfernt,
5. und arc-reduce darauf ausgeführt. Diese Funktion testet für jeden Wert xi in X (der Quellvariable), ob irgendein Wert in Y das Constraint erfüllt. Falls ja, bleibt xi in der Domain enthalten, ansonsten wird xi entfernt. Der aktualisierte Wertebereich wird zurückgeliefert.

[2] Englischer Begriff ist „strong ... consistency“

6. Ein leerer Wertebereich bedeutet eine Inkonsistenz. Der Algorithmus terminiert mit einem failure, Dx´ kann als Erklärung für das Scheitern heran
7. Aktualisierung der Queue: Wenn der Wertebereich von x in Schritt 5. eine Änderung erfahren hat, werden allen Kanten nach x erneut in die Queue eingefügt, und zwar per Mengenvereinigung (damit eine Kante nicht mehrfach in Q enthalten ist) - mit Ausnahme der Kante (y,x), denn diese ist von der Domainverkleinerung von x nicht betroffen.
8. Der Wertebereich von x wird auf die in arc-reduce aktualisierte Domain angepasst.
9. Ende der Schleife
10. Nachdem alle Kanten abgearbeitet sind (und der Algorithmus nicht vorzeitig in Zeile 6. terminierte), ist das CSP im 2-konsistenten Zustand. Die aktualisierten Wertebereiche aller Variablen werden zurückgeliefert.

A.K. Mackworth: Consistency in networks of relations. *Artificial Intelligence*, 8, S. 99-118, 1977.

43 2-Konsistenz mit dem AC-3-Algorithmus berechnen

Zu Beginn liegen die 4 gerichteten Kanten (von links nach rechts zu lesen) {A→B, B→A, B→C, C→B} in der Queue. Dargestellt in der Bearbeitungsreihenfolge in der Tabellenspalte 2. Die aktuellen Wertebereiche stehen in den Spalten D_A bis D_C.

Bei Eintritt des AC-Algorithmus in die Schleife (Tabellezeile #0) sind alle Wertebereiche noch initial (unverändert). Nachfolgend sind die Iterationen der Schleife (Spalte 1) aufgelistet:

1. A→B: Der Wert 0 in D_A ist nicht konsistent mit irgendeinem Wert in D_B, die anderen hingegen schon. Also wird D_A auf {1,2,3} verkleinert. Alle Kanten nach A (mit Ausnahme von B→A) müssen nun in die Queue eingefügt werden. Dies hat jedoch keinen Effekt, weil keine Kante (außer B→A) nach A führt.
2. B→A: Keine Veränderung für B im Test gegen die aktuellen Werte der Variable A (dargestellt als „k.V.“).
3. B→C: schränkt den Wertebereich von B auf {1,2} ein. Nun wird die Kante A→B in die Queue erneut eingefügt (in Zeile #5).
4. C→B: D_C = {2,3} wird verändert, aber es führen keine Kanten nach C (außer der inversen B→C). Also keine Veränderung in der Queue.
5. A→B: wird zum zweiten Mal untersucht. Aus dem Wertebereich von A wird die 3 entfernt.

Der Algorithmus terminiert, weil in der nächsten Iteration die Queue leer ist. Denn es führen keine Kanten aus der soeben betrachteten nach A.

$D_A = D_B = \{1,2\}$, $D_C = \{2,3\}$ stellt somit den 2-konsistenten Zustand des CSPs dar.

44 AC-3 hat polynomiellen Zeitaufwand

Laufzeitkomplexität ist $O(ned^3)$, wobei n die Zahl der Variablen, e die Zahl der binären Constraints und d die Größe des größten Wertebereichs ist. Zum Vergleich: Forward Checking: $O(nb)$)

Sei

- b = die Größe des Wertebereichs der größten Variablen,
- n = die Anzahl der Variablen.

Dann existieren höchstens n^2 Zweige – nämlich maximal zwischen jedem Paar von Variablen (wir betrachten ausschließlich binäre Constraints). Für jeden Zweig müssen höchstens b^2 Vergleiche (Tabellenzugriffe) durchgeführt werden – nämlich im Worst Case jede mögliche Wertekombination. Jeder Zweig wird höchstens b mal betrachtet – nämlich im Worst Case, wenn in jedem Schleifendurchlauf nur jeweils ein Wert aus einer Domain entfernt wird. Also ergibt sich als Kostenaufwandsabschätzung: O(n^2 b^3).

Es gibt bessere Algorithmen zur Erzielung von 2-Konsistenz (z.B. AC-2001) – diese haben einen Laufzeitaufwand von O(n*b^2) und einen Speicherverbrauch von O(n*b).

Bemerkungen:

- AC-3 ist die 3. Variante einer Arc Consistency-Prozedur – es existieren noch 7 weitere, die teilweise noch effizienter arbeiten.
- AC-2001 (Bessiere, et al. 2005) optimiert die Filter-Methode: Für jeden Wert wird dessen „Support" berechnet. Das sind in jedem Constraint diejenigen Werte der anderen Variablen, die konsistent mit ihm sind. Immer wenn ein Wert durch das Filtern gelöscht wird, wird automatisch der Support um 1 dekrementiert. Werte mit 0 Support müssen entfernt werden, was zu einem verringerten Support weiterer Variablen führt. Der Vorteil dieser Vorgehensweise besteht darin, dass nicht jedes Mal alle Werte überprüft werden müssen.

45 Agenda: Zusammenfassung und Ausblick

46 Die Verfahren im Vergleich

(Vergleiche Folie #29 zur Erklärung der Situation)

Backtracking testet nur das Constraint zu der bereits belegten Variable – dieses liegt „in der Vergangenheit", weshalb man auch von „look back" spricht.

Forward Checking hingegen realisiert ein einfaches „look ahead", indem alle Constraints mit der gerade instanziierten Variable ausgewertet werden (im Beispiel 2 Constraints – die Reichweite ist also „2 Variablen in die Zukunft").

Echtes look ahead wird durch 2-Konsistenz erzielt. In diesem Fall werden die Auswirkungen der aktuell belegten Variable für alle „zukünftigen" Variablen betrachtet, d.h. alle Constraints werden in die Vorwärtsbetrachtung mit einbezogen.

Bemerkung: die drei Constraints zwischen den früher belegten Variablen und freien Variablen werden weder durch FC noch durch Look Ahead-Methoden geprüft. Der Grund dafür ist, dass diese Constraints bereits endgültig konsistent sind und nicht erneut ausgewertet werden müssen, weil die eine der beteiligten Constraintvariablen bereits auf einen einzigen Wert eingeschränkt ist (die belegte Variable). Sowohl Forward Checking als auch Arc Consistency hat dies bereits zum früheren Zeitpunkt sichergestellt – nämlich in der Propagierung, die sich der Variablenbelegung anschließt.

47 Fazit

Chronologisches Backtracking ist für schwierig zu lösende CSPs unpraktikabel. Backtracking in Verbindung mit Forward Checking ist zumeist effizienter als in Verbindung mit Zweikonsistenz, denn der relative geringe Aufwand für das partial look-ahead schlägt in der Regel den höheren Berechnungsaufwand für das full look ahead. Insbesondere, wenn das CSP aus vielen Ungleichheitsconstraints besteht, denn dann werden die Constraints ohne belegte Variablen ausgewertet, aber ohne einen Effekt. Hingegen macht die Verwendung von 2-Konsistenz Sinn, wenn stärkere Constraints (z.B. Gleichheit) existieren, denn dann ergibt sich aus dem höheren Konsistenzberechnungsaufwand stärkere Domaineinschränkungen und somit ein geringerer Verzweigungsgra und weniger Backtrackingpunkte.

Praxistipp:

1. 1-Konsistenz herstellen
2. gegebenenfalls 2-Konsistenz herstellen (aber nur einmal)
3. Suche mittels Backtracking + Forward Checking unter Einsatz von Heuristiken

48 Baumartige CSPs sind in polynomieller Zeit lösbar

Wir verwenden D als Wurzelknoten. Der Baum hat dann eine Tiefe von 3.

Eine Lösung wird gefunden nach 2 Durchläufen durch den Baum. Zuerst wird „halbe Kantenkonsistenz" erzielt durch Betrachtung der Kanten von den Blättern zur Wurzel. Dabei wird jede Variable einmal besucht. Die Blätter behalten ihre initialen Wertebereiche, aber nach oben finden dann die Constraintauswertungen statt. Das ergibt einen Aufwand von O(b N^2) (b = maximale Domaingröße; N = Anzahl der Variablen). Auf dem Weg

von den Blättern zur Wurzel kann eine Inkonsistenz festgestellt werden, wenn nämlich ein Wertebereich leer wird. Dann ist das CSP nicht lösbar. Ansonsten ist das CSP lösbar, denn jeder Knoten enthält ja nur solche Werte, die alle Constraints zu den Kindknoten erfüllen.

Nun wird der Baum von der Wurzel zu den Blättern schrittweise konsistent belegt.

49 Baumtransformation durch Entfernen echter Graphen reduziert die Komplexität

Der Graph oben links enthält ein weiteres Constraint C-B, das die Baumstruktur zerstört: {B,C,E} formen einen echten Constraintgraph.

Oben rechts: Betrachtet man das Teil-CSP {B,C,E} als *einen* Knoten, ergibt sich wiederum ein Baum. Dieser ist unten links notiert. Der grüne Knoten CSP-1 hat Verbindungen zu den Knoten D und G. Genau diese Knoten werden nun zu Eltern und Kind.

Nun ist der Baumalgorithmus in einer angepassten Form anwendbar:

1. Initial wird das CSP-1 gelöst. Es werden *alle* vollständigen und konsistenten Belegungen ermittelt. Dieser Aufwand ist exponentiell, aber nur in Bezug auf die k<N enthaltenen Variablen des CSP-1.
2. Die Bottom-Up-Phase des Baumalgorithmus findet statt. Der Zweig V(2,G)→CSP-1 wird ausgewertet und der Wertebereich der betroffenen Variable in CSP-1 entsprechend verkleinert. Von einer Domaineinschränkung in CSP-1 sind natürlich auch die anderen k-1 Variablen betroffen. Dies ist einfach zu ermitteln, weil die Lösungen ja bereits vorab berechnet wurden. Beim Zweig CSP-1→V(0,D) wird ebenfalls nur die in Constraintbeziehung mit V(0,D) stehende Variable genutzt.
3. Die Belegungsphase findet statt. Auch hier werden nur die betroffenen Variablen von CSP-1 verwendet.

Bemerkungen

- Die Ermittlung aller Lösungen von CSP-1 erfordern einen Aufwand von O(d^k) im Worst Case (k ist die Zahl der Variablen in CSP-1).
- Zusammen mit der Komplexität des Baumalgorithmus von O((N-k) d^2) (vergleiche Folie “Baumartige CSPs sind in polynomieller Zeit lösbar“) ergibt sich die Komplexität O((N-k+1) d^(k+2)).
- Kritisch ist also die Größe des CSP-1, denn die Zahl k der Variablen in CSP-1 steht im Exponenten.
- Das minimale k eines Graphen ist nicht in polynomieller Zeit zu finden. Daher kann ein CSP nicht garantiert mit polynomiellem Aufwand gelöst werden. Dies ist keine Überraschung, weil CSPs im Worst Case ja NP-schwer sind und daher keine polymomielle Lösungszeit haben.

50 Ausblick

- Constraintbibliotheken für viele Programmiersprachen verfügbar. Auch hoch performante Constraintsolver in nativen Sprachen existieren (ILOG, jetzt IBM).
- Constraintsprachen sind deklarativ (Constraint Logic Programming). Constraintprogramme oft sehr trivial: Z.B. Variablen und Constraints spezifizieren und dann einen Generate&Test-Dreizeiler schreiben, der das CSP löst. Intern verwendet der simple Algorithmus dann Forward Checking, etc.
- CLP(IR) ist so eine deklarative Sprache (Prolog-Erweiterung), die nicht nur finite Domains sondern auch algebraische Constraints (Gleichungen und Ungleichungen mit Variablen) beinhaltet. Intern arbeitet dann ein Simplex-Solver diese (Un)Gleichungen ab bzw. vereinfacht diese. (https://en.wikipedia.org/wiki/CLP(R)
- Constraint Handling Rules: (Prolog) erlaubt die Definition von Regeln, welche Constraints nach bestimmten Kriterien auswerten oder vereinfachen.
- Constraintoptimierung optimiert eine Zielfunktion. Basiert zu großen Teilen auf OR-Methoden wie Lineare Programmierung.

Literatur

R. Bartak: Online Guide to Constraint Programming. http://ktiml.mff.cuni.cz/~bartak/constraints/index.html

R. Bartak: Constraint Propagation and Backtracking Based Search.
http://ktiml.mff.cuni.cz/~bartak/downloads/CPschool05notes.pdf

Freuder, E. C. (1982). A Sufficient Condition for Backtrack-Free Search. Journal of the ACM.
doi:10.1145/322290.322292

V. Kumar: Algorithms for Constraint Satisfaction Problems: A Survey. *AI Magazine 13(1), S.* 32-44, 1992.

A.K. Mackworth: Consistency in networks of relations. *Artificial Intelligence*, 8, S. 99-118, 1977.